

Chapitre 1 : Mesure des performances

Objectifs

À la fin du chapitre, vous serez capable de :

- Comprendre ce qu'est l'**évaluation des performances**.
- Identifier les **facteurs fondamentaux** influençant la performance d'une machine.
- **Calculer la fréquence d'horloge**.
- Déterminer le **critère principal d'évaluation** des performances : **le temps d'exécution**.

1/Introduction

Depuis la création de l'ordinateur, on cherche à améliorer :

- La **vitesse de calcul**,
- Le **temps d'accès mémoire**,
- Les **opérations d'E/S (Entrée/Sortie)**.

Avant 1945 → machines **mécaniques**, lentes et peu fiables.

Après 1945 → apparition de l'électronique (signaux électriques → vitesse proche de la lumière).

2/Pourquoi mesurer les performances ?

- Identifier les **faiblesses** d'un ordinateur.
- **Comparer** ou **calculer** la performance d'un processeur.
- **Optimiser / augmenter** la performance globale du système.

3/Caractéristiques d'un processeur

1. Côté logiciel (Software)

- **Jeu d'instructions** :
 - **CISC** → instructions complexes, plusieurs cycles. (Ex : x86)
 - **RISC** → instructions simples, peu de cycles. (Ex : PowerPC)
- **Compilateur** :
 - La **génération de code** doit être **optimisée** pour de meilleures performances.

2. Côté matériel (Hardware)

- **Complexité de l'architecture** : plus de transistors = plus d'opérations/seconde.
- **Fréquence d'horloge** : plus elle est élevée, plus le processeur exécute d'instructions rapidement.

- **Technologie des semi-conducteurs, circuits, mémoires, caches** influencent aussi la performance.

4/Évaluation des performances

♦ Définitions :

- **Côté utilisateur** : temps nécessaire pour exécuter une tâche.
- **Côté système** : nombre de tâches exécutées par unité de temps (**débit**).

Améliorer le temps de réponse améliore aussi le débit, mais augmenter le débit n'améliore pas toujours le temps de réponse.

4.1/Critère principal : le temps

Formule générale :

$$T = N_{instr} \times CPI \times \text{Cycle d'horloge}$$

où :

- N_{instr} = nombre d'instructions exécutées (Ic)
- CPI = nombre moyen de cycles par instruction
- τ = durée d'un cycle d'horloge

4.2/Facteurs fondamentaux de performance

1. Cycle processeur

- Le processeur est piloté par une **horloge** de période τ . L'inverse du cycle donne la fréquence d'horloge (cycle processeur).
- La **fréquence d'horloge** (f) est donnée par :

L'**horloge** contrôle le rythme des opérations du processeur.

$$f = \frac{1}{\tau}$$

τ (**temps de cycle**) correspond à la durée d'un cycle d'horloge.

La **fréquence** indique le nombre de cycles exécutés par seconde (ex. : 1 GHz = 1 milliard de cycles/s).

Exemple : 1 GHz $\leftrightarrow$ période = 1 ns

Remarque :

Ic dépend du **jeu d'instructions** et du **compilateur** utilisé.

CPI dépend du **chemin de données** et du **jeu d'instructions**.

τ dépend de l'**architecture matérielle** du processeur.

2. Compteur d'instructions (Ic)

Nombre total d'instructions dans le programme.

3. CPI (Cycles Par Instruction)

Nombre moyen de cycles nécessaires pour exécuter une instruction.

4. Temps CPU (formule principale)

$$T = Ic \times CPI \times \tau = \frac{Ic \times CPI}{f} \quad (\text{Eq.2})$$

5. Cycle mémoire

Est le temps d'accomplissement d'une référence mémoire. En général le cycle mémoire est k fois plus grand que le cycle processeur (τ). Eq...(2) devient

avec :

- p = nb de cycles pour décoder/exécuter une instruction
- m = nb de références mémoire nécessaires

$$T = Ic \times (p + m \times k) \times \tau \quad (\text{Eq.3})$$

6. Nombre total de cycles

- Soit C le nombre total de cycle d'horloge nécessaires pour exécuter un programme donné. Donc le temps CPU donné par eq..2 est:

$$T = C \times \tau = \frac{C}{f} \quad (\text{Eq.4})$$

$$CPI = \frac{C}{Ic} \quad (\text{Eq.5})$$

7. Vitesse MIPS (Million Instructions Per Second)

- La vitesse du processeur est mesurée en MIPS (Million Instructions Per Second). Elle est calculée comme suit:

$$MIPS = \frac{Ic}{T \times 10^6} = \frac{f}{CPI \times 10^6} = \frac{f \times Ic}{C \times 10^6} \quad (\text{Eq.7})$$

D'où :

$$T = Ic \times \frac{10^{-6}}{MIPS} \quad (\text{Eq.8})$$

8. Rendement (Throughput rate, Wp)

Nombre de programmes exécutés par unité de temps.

$$W_p = \frac{f}{I_c \times CPI} \quad (\text{Eq.9})$$

$$W_p = (MIPS) \times \frac{10^6}{I_c}$$

Unité : programmes/seconde

Remarques importantes

- I_c dépend du **jeu d'instruction** et du **compilateur**.
- CPI dépend du **chemin de données** et du **jeu d'instruction**.
- τ dépend de l'**architecture matérielle**.
- Le **temps d'exécution (T)** reste le **meilleur indicateur** global de performance.

Résumé des principales formules

Nom	Formule	Équation
Fréquence d'horloge	$f = \frac{1}{\tau}$	
Temps CPU principal	$T = I_c \times CPI \times \tau = \frac{I_c \times CPI}{f}$	(Eq.2)
Temps avec mémoire	$T = I_c \times (p + m \times k) \times \tau$	(Eq.3)
Temps CPU (cycles totaux)	$T = C \times \tau = \frac{C}{f}$	(Eq.4)
Moyenne des cycles	$CPI = \frac{C}{I_c}$	(Eq.5)
Vitesse MIPS	$MIPS = \frac{f}{CPI \times 10^6} = I_c / (T \times 10^6) = (f \times I_c) / (C \times 10^6)$	(Eq.7)
Temps via MIPS	$T = I_c \times \frac{10^{-6}}{MIPS}$	(Eq.8)
Rendement	$W_p = \frac{f}{I_c \times CPI} = (MIPS) \times ((10^6) / (I_c))$	(Eq.9)

Chapitre 2 : Programmation en langage de bas niveau (Assembleur)

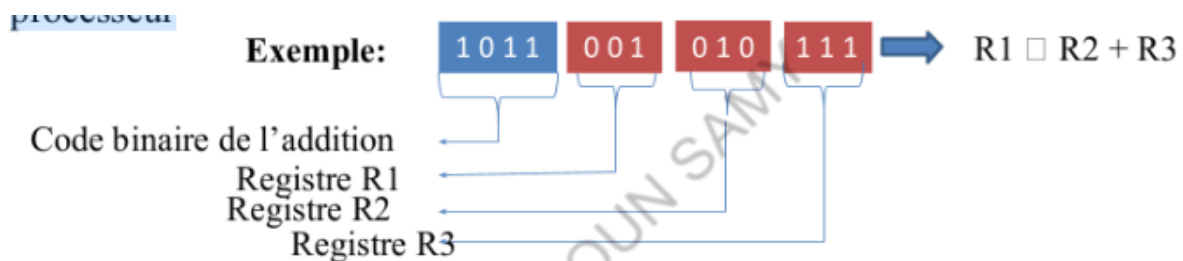
Introduction

Les performances dépendent du **matériel** et de la **manière dont le programme l'utilise**. Pour exploiter au mieux un processeur, le code doit être **efficace et proche du matériel**. Ce chapitre vise à comprendre **l'exécution des instructions** et l'usage de **l'assembleur** pour interagir directement avec le processeur.

On y apprendra à **manipuler les registres, gérer la mémoire, interpréter les instructions machine** et **analyser leur impact sur les performances**.

Introduction: Architecture de base d'un microprocesseur

Programmer en assembleur nécessite de connaître l'architecture du processeur, ce qui permet de coder en langage machine ou en langage assembleur, des langages de bas niveau. L'architecture externe définit ce qu'un programmeur doit connaître pour coder directement en langage du processeur. Le langage machine est le code binaire spécifique à chaque processeur.



Le langage assembleur est similaire au langage machine, mais utilise des **mnémoniques** pour les instructions et les opérandes (ex. : **Add R1, R2, R3**).

L'architecture externe d'un processeur concerne les éléments suivants :

Un programmeur doit connaître :

- **Registres visibles** : nombre de registres et taille des registres manipulables par le programme.
- **Adressage mémoire** : taille des bus de données du processeur et d'adresse du processeur, et modes d'adressage de la mémoire.
- **Jeu d'instructions** : format de l'instruction et nombre d'instructions disponibles.
- **Les mécanismes de traitement des interruptions et exceptions** : lignes d'interruption, priorité d'interruptions et l'emplacement du gestionnaire.

Pourquoi connaître l'architecture d'un processeur?

Connaître l'architecture d'un processeur permet de programmer directement en langage machine.

Avantages des langages de bas niveau :

- **Contrôle total du matériel** (registres, mémoire, instructions).

- **Exécution rapide** : grâce à un code optimisé.
- **Faible consommation mémoire** : utile pour systèmes embarqués.
- **Bonne compréhension du fonctionnement interne** pour optimisation et débogage.
- **Pertinent pour les systèmes critiques** nécessitant performance et fiabilité.

Exemple : un programme "Hello world" occupe 15 389 octets en C contre 23 octets en assembleur. Pour faciliter la programmation, on utilise le langage **assembleur**, plus convivial que le code machine.

Étude de cas: Processeur Mips R3000

Présentation du processeur MIPS R3000

- Le processeur **MIPS-R3000** est un processeur **32 bits** industriel conçu dans les années 80.
- Utilisé initialement dans des **stations de travail, serveurs et systèmes tolérants aux fautes**, puis dans un large éventail d'applications, y compris les **systèmes embarqués** (consoles de jeux, PDA).
- Il utilise un **jeu d'instructions RISC** (Reduced Instruction Set Computer).
- Microprocesseur moderne avec **protection et multitâche**.
- Dispose de **deux modes de fonctionnement** :

- **Superviseur** : mode protégé pour les programmes système.
- **Utilisateur** : mode pour les programmes des utilisateurs.

Chaque mode possède ses **propres registres visibles**.

Registres visibles du MIPS R3000 :

a) Registres non protégés

- **32 registres à usage général (\$R0 à \$R31)** :
 - Stockent les opérandes pour les opérations arithmétiques et logiques.
 - **\$R0** vaut toujours 0.
 - **\$R31** sert à sauvegarder l'adresse de retour lors d'un appel de procédure.
- **Compteur Programme (\$PC)** : contient l'adresse de l'instruction en cours, modifié par toutes les instructions.
- **Registres HI et LO** : utilisés pour stocker les résultats des multiplications et divisions.

Les 32 registres (registre à usage général) sont donnés dans le tableau suivant :

Nom	Numéro de registre	Usage
\$zero	0	Dédié à la constante 0
\$v0-\$v1	2-3	Dédié à l'évaluation des expressions
\$a0-\$a3	4-7	Stockage des arguments lors des appels de fonctions
\$t0-\$t7	8-15	Registres temporaires
\$s0-\$s7	16-23	Registres sauvegardés
\$t8-\$t9	24-25	Registres temporaires supplémentaires
\$gp	28	Pointeur global
\$sp	29	Pointeur de pile
\$fp	30	Pointeur de "frame"
\$ra	31	Pointeur d'adresse retour

b) Registres protégés

-32 registres (0 à 31) accessibles uniquement par **instructions privilégiées**, appartenant au **coprocesseur système**.

Principaux registres :

- **SR (Status Register)** : indique le mode (superviseur/utilisateur) et masque les interruptions.
- **CR (Cause Register)** : précise la cause d'une interruption ou exception.
- **EPC (Exception Program Counter)** : adresse de retour après interruption ou adresse de l'instruction fautive en cas d'exception.
- **BAR (Bad Address Register)** : contient l'adresse incorrecte lors d'une exception de type « adresse illégale ».

Organisation de la mémoire :

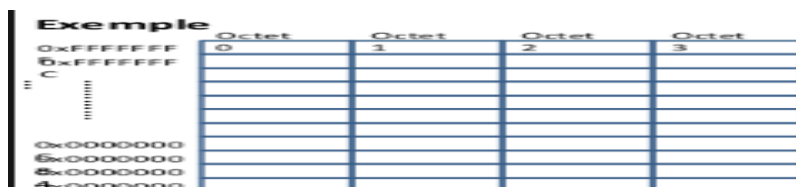
L'organisation de la mémoire varie selon le processeur, notamment la **taille des bus de données et d'adresses**.

La **taille du bus de données** détermine celle du mot mémoire :

- 8 bits → bus 8 bits
- 16 bits → bus 16 bits
- 32 bits → bus 32 bits
- 64 bits → bus 64 bits

Organisation mémoire de MIPS-R3000

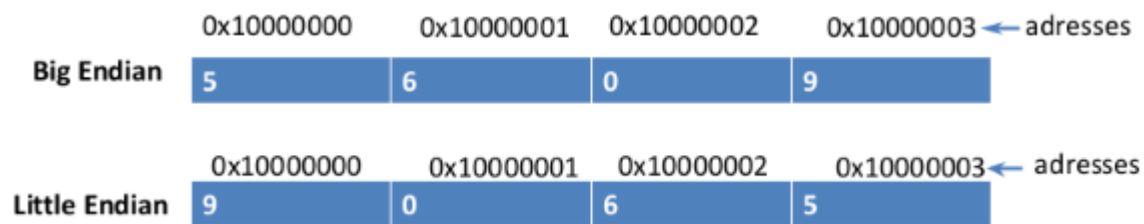
- Le MIPS R3000 a un **bus de données et d'adresses de 32 bits** et une **mémoire organisée par octets**.
- Il peut charger des mots de **32 bits, 16 bits ou 8 bits**, les mots plus larges étant stockés comme une **séquence d'octets**.
- L'**adresse d'un mot** correspond à celle du **premier octet**.
- La mémoire est vue comme un **tableau d'octets** contenant données et instructions.
- Toutes les **instructions** sont sur 32 bits.
- Les **données** peuvent être : 32 bits (word), 16 bits (half word), 8 bits (byte).
- Les **adresses doivent être alignées** : multiples de 4 pour les mots, multiples de 2 pour les demi-mots.



Big Endian vs Little Endian

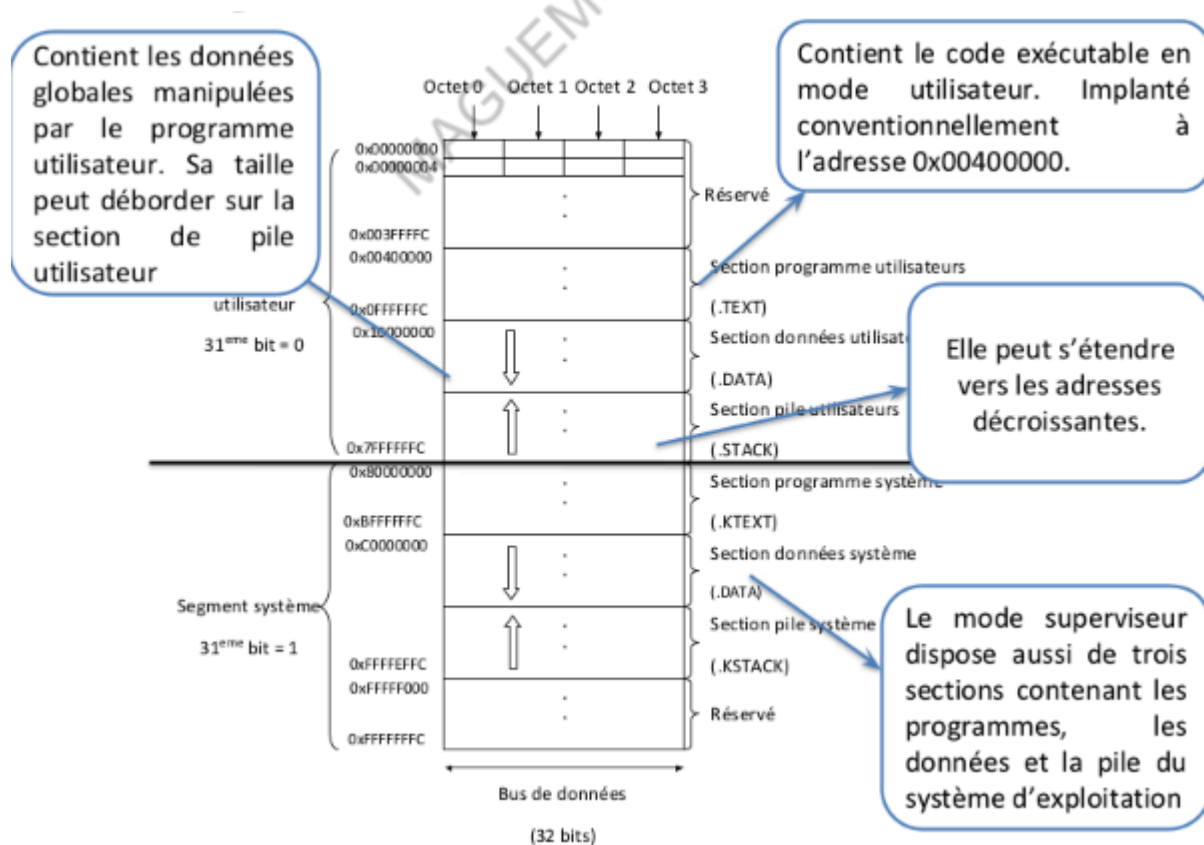
- **Endianess** définit l'ordre de stockage des octets lorsqu'un objet occupe plusieurs octets.
- **Big Endian** : l'adresse correspond à l'**octet de poids fort**, les octets de poids faible suivent.

- **Little Endian** : l'adresse correspond à l'**octet de poids faible**, les octets de poids fort suivent.
- Exemple : $X = 0x5609$ stocké à l'adresse $0x10000000$.



MIPS R3000 utilise la représentation Little Endian

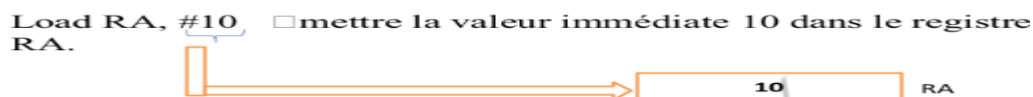
- La mémoire est divisée en **2 segments** selon le **bit de poids fort (31ème bit)** :
 - 0 → segment utilisateur
 - 1 → segment système
- En **mode superviseur**, les deux segments sont accessibles.
- En **mode utilisateur**, seul le segment utilisateur est accessible.
- Une **exception** est générée si une instruction en mode utilisateur tente d'accéder au segment système.
- Un programme contient généralement :
 - **Section instructions**
 - **Section données**
 - **Pile** pour les appels de procédures et fonctions
- La mémoire du MIPS-R3000 est organisée selon cette structure.



Modes d'adressage de la mémoire :

- L'accès à la mémoire se fait via une **adresse spécifiée dans l'instruction**.
- Les principaux **modes d'adressage** sont :
 - immédiat
 - direct
 - indirect
 - registre
 - indirect par registre
 - basé (ou par déplacement)
 - indexé
 - basé indexé
 - relatif

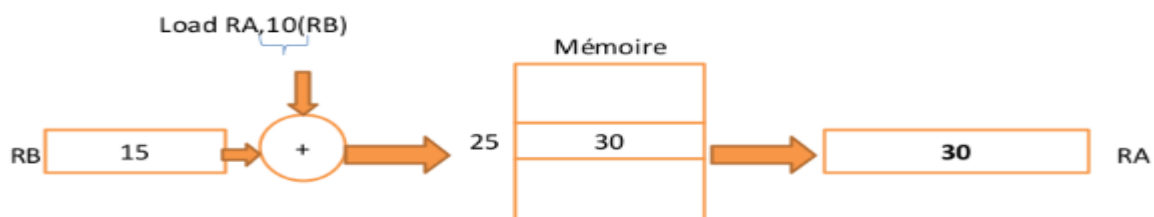
Adressage immédiat : l'instruction contient directement la **valeur de l'opérande**, permettant de manipuler des constantes sans accéder à la mémoire.



Adressage par registre : l'opérande est dans un **registre interne** du processeur, sans accès à la mémoire.



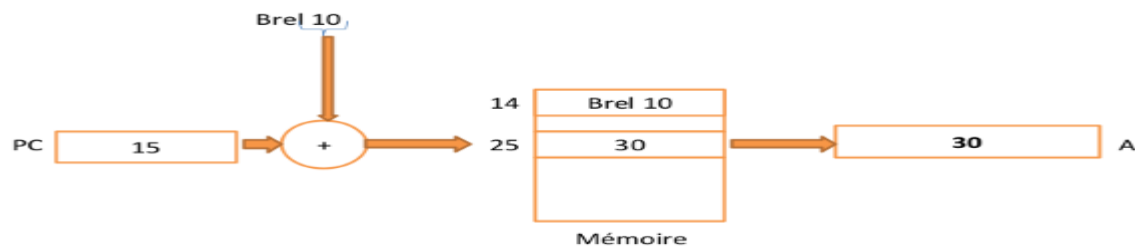
Adressage basé : l'adresse de l'opérande est calculée à partir d'une **base** dans un registre et d'un **déplacement** dans l'instruction.



Exemple:

Un tableau, par exemple peut être adressé par un registre qui indique la base du tableau, et un déplacement qui fait référence à un élément à l'intérieur du tableau.

Adressage relatif : l'adresse de l'opérande est calculée à partir du **compteur d'instructions** plus un **déplacement**, souvent utilisé pour les branchements.

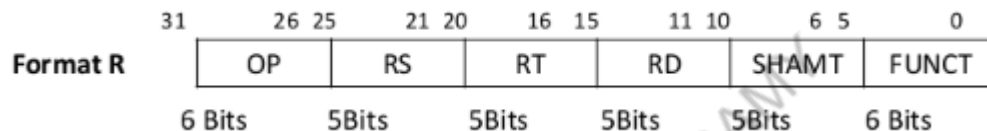


Le MIPS-R3000 utilise les modes d'adressage : **immédiat, par registre, basé et relatif**.

Formats d'instructions :

- Les instructions peuvent avoir **1, 2 ou 3 adresses** selon le nombre d'opérandes.
- Toutes les instructions du MIPS-R3000 font **32 bits**. Il existe 3 formats d'instructions:
 - a) Format R** : utilisé pour les instructions avec **2 registres sources** et **1 registre destination**.

Le format est donné par la figure suivante:

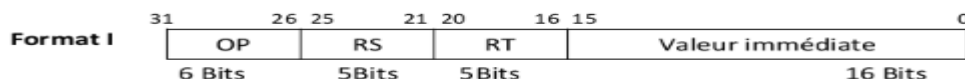


- OP** : code de l'opération (addition, soustraction, etc.), FUNCT distingue les instructions ayant le même OP.
- RS** : premier registre source (1er opérande).
- RT** : deuxième registre source (2e opérande).
- RD** : registre destination (où le résultat est stocké).
- SHAMT** : nombre de décalages pour les instructions de décalage.
- FUNCT** : précise l'instruction lorsque plusieurs partagent le même OP.

Code_operation Résultat, Opérande1, Opérande2.

Exemple: Add A, B, C

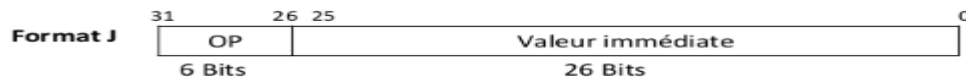
b) Format I : utilisé pour les accès mémoire (lecture/écriture...), les instructions avec opérande immédiat et les branchements conditionnels; la valeur immédiate est limitée à **16 bits**.



Code_operation Destination, Source.

Exemple: Load A, B

c) Format J : utilisé pour les **branchements inconditionnels à longue distance**, avec une adresse de **26 bits**.



Exemple: J 100

- Le MIPS est un processeur **Load/Store** : seules deux instructions accèdent à la mémoire — **Load** (charger) et **Store** (enregistrer).
- Toutes les autres instructions (arithmétiques, logiques...) utilisent uniquement les **registres** pour les opérandes et le résultat.
- Cette approche RISC est utilisée car **l'accès à la mémoire est lent**, tandis que les opérations sur registres sont plus rapides.

Exemple LW R5, 126(R8) :

- **Signification** : charger R5 avec le contenu de l'adresse R8 + 126.
- **Format I** : OP (6 bits) | RS (5 bits) | RT (5 bits) | Valeur immédiate (16 bits)
 - OP = 100011
 - RS (R8) = 01000
 - RT (R5) = 00101
 - Valeur immédiate = 0000000001111110
- **Code machine** : 10001101000001010000000001111110 → hex : 0x8D05007E

Exercice :

- **Instruction machine** : 0x00642820

Correction :

1. Convertir en binaire : 0x00642820 → 000000 00011 00100 00101 00000 100000
2. Identifier le format : **Format R** (opcode = 000000)
3. Décomposer les champs :
 - RS = 00011 → R3
 - RT = 00100 → R4
 - RD = 00101 → R5
 - SHAMT = 00000
 - FUNCT = 100000 → ADD

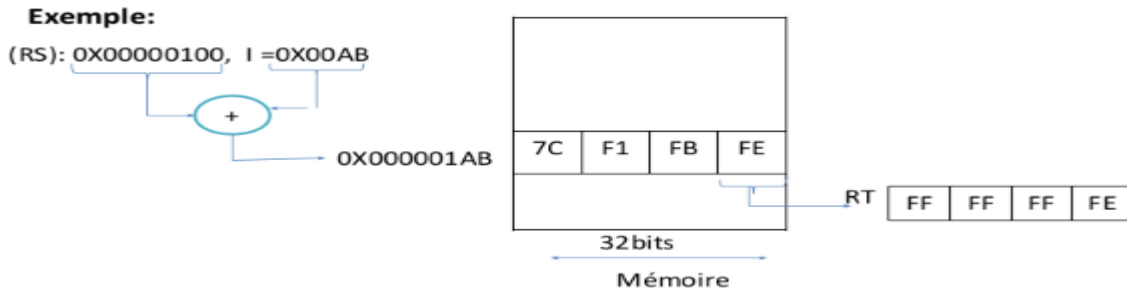
Instruction assembleur correspondante :

ADD R5, R3, R4

Etude de quelques instructions :

a) Instructions d'accès à la mémoire

LB RT, I(RS) : charge 1 octet de mémoire à l'adresse RS + I dans RT, en étendant le signe sur les octets de poids fort.



- **LBU RT, I(RS) :**
 - Charge **1 octet** depuis l'adresse **RS + I** dans le registre **RT**.
 - **Signe ignoré** : les octets de poids fort sont mis à zéro.
 - Exemple : si la valeur mémoire = **0xFE**, alors **RT = 0x000000FE**.
- **LH RT, I(RS) :**
 - Charge **2 octets** depuis **RS + I** dans les octets de poids faible de **RT**.
 - **Signe étendu** : les octets de poids fort prennent le signe de la valeur chargée.
 - Exemple : **RT = 0xFFFFFBFE**.
- **LHU RT, I(RS) :**
 - Charge **2 octets** depuis **RS + I** dans les octets de poids faible de **RT**.
 - **Signe ignoré** : les octets de poids fort sont mis à zéro.
 - Exemple : **RT = 0x0000FBFE**.
- **LW RT, I(RS) :**
 - Charge **1 mot (4 octets)** depuis **RS + I** dans **RT**.
 - Exemple : **RT = 0x7CF1FBFE**.

Remarque :

- Les instructions **SB, SH, SW** fonctionnent de manière similaire mais servent à **enregistrer des données en mémoire**.

b) Instructions arithmétiques et logiques

- **ADD RD, RS, RT** : addition avec détection de débordement → exception si overflow.
- **ADDU RD, RS, RT** : addition sans débordement → overflow ignoré.
 - **Remarque** : SUB et SUBU font pareil mais pour la soustraction.
- **MULT RS, RT** : multiplication signée → résultat faible dans **LO**, résultat fort dans **HI**, exception si overflow.
- **MULTU RS, RT** : multiplication non signée → overflow ignoré.
- **DIV RS, RT** : division signée → **LO = quotient**, **HI = reste**.
- **DIVU RS, RT** : division non signée.
- **OR RD, RS, RT** : OU logique bit à bit entre RS et RT → résultat dans RD.
- **AND RD, RS, RT** : ET logique bit à bit entre RS et RT → résultat dans RD.

c) Instructions manipulant des valeurs immédiates

- Les instructions immédiates (format I) ne peuvent manipuler que des constantes ou adresses **16 bits**.

- Pour charger une constante ou adresse **32 bits**, il faut combiner plusieurs instructions.

Instructions principales :

- **LUI RT, I** : charge la valeur immédiate **I** dans les **16 bits de poids fort** de **RT**, et met les 16 bits de poids faible à 0.
- **ORI RT, RS, I** : effectue un OU logique entre le contenu de **RS** et la valeur immédiate **I**, résultat dans **RT**.

Exemple :

Pour stocker **0x12FE2C3B** dans **\$R5** :

1. **LUI \$R5, 0x12FE** → **\$R5 = 0x12FE0000**
2. **ORI \$R5, \$R5, 0x2C3B** → **\$R5 = 0x12FE2C3B**

d) Instructions de branchement

- Permettent de réaliser des conditions (**if-then-else**) et des boucles.
- Le branchement peut être **relatif** (par rapport à l'instruction courante) ou **absolu** (à une adresse précise).

Principales instructions :

- **BEQ RS, RT, I** : si **RS = RT**, sauter à l'instruction à distance **I**, sinon continuer.
- **BGTZ RS, I** : si **RS > 0**, sauter à l'instruction à distance **I**, sinon continuer.
- **J I** : saut absolu à l'instruction d'adresse **I**.
- **JAL I** : saut à l'instruction d'adresse **I** et sauvegarde l'adresse suivante dans **\$R31** (appel de sous-programme).
- **JR RS** : saut à l'adresse contenue dans **RS** (retour de procédure).

Exercice:

Ecrire un programme assembleur MIPS qui récupère dans le registre R10, la plus grande valeur se trouvant dans les registres R11 et R12.

Programme C:

```
If (R11 >= R12) {
  R10=R11
} else {
  R10=R12}
```

Méthode 1 :

```
SUB R13, R11, R12  # R13 = R11 - R12
BLTZ R13, else     # si R13 < 0 (R11 < R12), aller prendre R12
ADD R10, R11, R0    # sinon, R10 = R11
J end
else:
  ADD R10, R12, R0  # R10 = R12
end:
```

Méthode 2 :

```
SUB R13, R12, R11  # R13 = R12 - R11
```

```

BGEZ R13, if # si R13 >= 0 (R12 >= R11), prendre R12
ADD R10, R11, R0 # sinon, R10 = R11
J end

```

if:

```

ADD R10, R12, R0 # R10 = R12

```

end:

Type de données manipulées par le processeur MIPS

1) Entiers et caractères:

- Les entiers en MIPS sont en **complément à 2** et peuvent avoir différentes tailles.
- Les caractères sont codés en **ASCII** sur 1 octet et peuvent être vus comme des entiers non signés sur 8 bits.

- La table suivante résume les types utilisés dans les langages évolués(C par exemple) et leur correspondance en assembleur:

Type C	Type MIPS	Nombre d'octets
Long long	Dword(64bits)	8(2accès mémoire)
Long(ou int)	Word(32bits)	4(1accès mémoire)
Short	Halfword(16bits)	2(1accès mémoire)
Char	Byte	1(1accès mémoire)

2) Nombre réels (float en C):

- Les MIPS ont un **coprocesseur pour les calculs flottants** (coprocesseur 1) avec **32 registres de 32 bits** (f0 à f31).
- Les nombres flottants suivent la **norme IEEE-754**, en simple ou double précision.
- La **double précision (64 bits)** utilise deux registres consécutifs.
- Des instructions spécifiques permettent les **opérations sur les flottants**.

3) Chaîne de caractères:

- Une **chaîne de caractères (string)** est une suite de caractères.
- Trois représentations possibles :
 - **Pascal** : premier octet = taille de la chaîne.
 - **C/Java** : dernier octet = caractère nul (**0**) indiquant la fin.
 - Avec variable séparée indiquant la taille (rarement utilisé).
- En **assembleur**, on utilise généralement la **représentation C**, plus simple à manipuler.

Chapitre 3 : Ecriture de programmes en langage d'assemblage MIPS

- L'assembleur MIPS sert à écrire des programmes pour le **processeur MIPS-R3000**.
- Les systèmes MIPS définissent des **conventions pour l'usage des registres**.
- Ces conventions **ne sont pas obligatoires**, mais permettent à plusieurs programmeurs de créer des programmes **compatibles et cohérents**.

Conventions d'utilisation des registres MIPS :

- **\$at (1), \$k0 (26), \$k1 (27)** : réservés au système et à l'assembleur, **ne pas utiliser**.
- **\$a0 - \$a3 (4-7)** : servent à passer les 4 premiers arguments aux fonctions ; les arguments supplémentaires passent par la pile.
- **\$v0, \$v1** : utilisés pour renvoyer les valeurs de retour.
- **\$t0 - \$t9 (8-15, 24, 25)** : registres temporaires, **non préservés** lors d'un appel de fonction.
- **\$s0 - \$s7 (16-23)** : registres sauvegardés, la fonction doit préserver et restaurer leur contenu.
- **\$gp (28)** : pointe vers le segment de données globales.
- **\$sp (29)** : pointe vers la prochaine position libre de la pile.
- **\$fp (30)** : pointe vers les données locales dans la pile.
- **\$ra (31)** : sauvegarde l'adresse de retour lors d'un appel de procédure.

Directives d'assemblage et pseudo-instructions MIPS

- **Directives d'assemblage** : commandes pour l'assembleur afin de générer le code objet.
- **Pseudo-instructions** : macros qui remplacent un ensemble d'instructions réelles pour faciliter certaines tâches.

1) Directives :

- **.data** : les éléments suivants sont enregistrés dans la section donnée (mémoire utilisateur 0x10000000 → 0x7FFFFFFF).
- **.text** : les éléments suivants sont enregistrés dans la section programme (code).
- **.byte b1,b2,...bn** : enregistre n octets en mémoire.
- **.half h1,h2,...hn** : enregistre n demi-mots en mémoire.
- **.word w1,w2,...wn** : enregistre n mots en mémoire.
- **.ascii "chaîne"** : enregistre la chaîne de caractères en mémoire.
- **.asciiz "chaîne"** : enregistre la chaîne et ajoute le caractère nul 0 à la fin.
- **.float f1,f2,...fn** : enregistre les nombres flottants en mémoire.
- **.double d1,d2,...dn** : enregistre des doubles-mots en mémoire.

2) Pseudo-instructions :

- **LA RT, Etiquette** : charge RT de l'adresse de l'étiquette (32 bits si nécessaire).
 - Equivalent à : **LUI RT, Adresse(31-16) de l'étiquette**

ORI RT, Adresse(15-0) de l'étiquette

LI RT, I : charge RT avec la valeur immédiate I (32 bits possible).

MOVE RD, RS : copie le contenu de RS dans RD.

- Equivalent à : **OR RD, RS, \$R0**

MUL RD, RS, RT : multiplie RS et RT, stocke le résultat dans RD.

- Equivalent à : **MULT RS, RT**

MFLO RD # RD <- LO

Simulateur MIPS (SPIM) :

- **SPIM** : logiciel qui simule l'exécution de programmes assembleur MIPS.
- Permet de **vérifier et tester** les programmes et de voir leurs résultats.

Format d'un programme sous SPIM :

```
.data # section données : déclarer toutes les données
.byte .....
.word .....
.text # section programme : écrire les instructions
.globl main # étiquette obligatoire au début
main: # instructions du programme .....
```

Exemples de programmes assembleur MIPS

1/ Addition de deux entiers

Objectif : Additionner deux valeurs 14 et 15, stockées dans les variables **A** et **B**, et sauvegarder le résultat dans **C**.

```
.data # Section données
A: .word 14 # Premier entier
B: .word 15 # Deuxième entier
C: .word 0 # Entier pour le résultat
.text # Section programme
.globl main # Point d'entrée du programme
main:
    la $t0, A # Charger l'adresse de A dans $t0
    lw $t1, 0($t0) # Charger le contenu de A dans $t1
    lw $t2, 4($t0) # Charger le contenu de B dans $t2
    add $t3, $t1, $t2 # Additionner $t1 et $t2, résultat dans $t3
    sw $t3, 8($t0) # Sauvegarder le résultat dans C
    jr $ra # Retour au système
```

Points clés :

- Les **variables** sont déclarées dans **.data** :
 - **A**: adresse du premier octet du mot contenant 14.

- **B**: adresse du mot contenant 15 ($B = A + 4$).
 - **C**: adresse du mot pour le résultat ($C = A + 8$).
- L'instruction **la \$t0, A** est une pseudo-instruction :
 - Elle charge l'adresse de A dans \$t0.
 - L'assembleur la traduit en :

```
lui $t0, partie_haute_A
ori $t0, $t0, partie_basse_A
```

Pour les **opérations arithmétiques**, tous les opérandes doivent être dans des registres.

- \$t1 = premier opérande (A)
- \$t2 = deuxième opérande (B)
- \$t3 = résultat

L'adresse de chaque opérande est calculée par **déplacement par rapport à la base** :

- Premier opérande : adresse de base ($0(\$t0)$)
- Deuxième opérande : $4(\$t0)$
- Résultat : $8(\$t0)$

Le programme commence toujours par **.globl main** et se termine par **jr \$ra**, pour permettre au simulateur SPIM de gérer le retour au système.

Addition des éléments d'un tableau de 10 entiers

Objectif : Calculer la somme des éléments du tableau $T = [1, 2, \dots, 10]$ et stocker le résultat dans **res**.

```
.data
T: .word 1,2,3,4,5,6,7,8,9,10 # Tableau de 10 entiers
res: .word 0 # Mot pour stocker le résultat
.text
.globl main
main:
    and $t4, $t4, $zero # Initialiser somme = 0
    la $t0, T # Charger l'adresse de base du tableau dans $t0
    li $t1, 10 # Compteur = nombre d'éléments

boucle:
    lw $t2, 0($t0) # Charger l'élément courant dans $t2
    add $t4, $t4, $t2 # Ajouter l'élément à la somme
    addi $t1, $t1, -1 # Décrémenter le compteur
    beq $t1, $zero, fin # Si compteur = 0, sortir de la boucle
    addi $t0, $t0, 4 # Passer à l'élément suivant (4 octets par mot)
    j boucle # Reboucler

fin:
    la $t0, res # Charger l'adresse du résultat
    sw $t4, 0($t0) # Stocker la somme dans res
```

jr \$ra # Retour au système

Points clés :

- **Déclaration des données :**
 - `.word` pour le tableau et le résultat.
- **Initialisation :**
 - `$t4` = somme partielle, mis à zéro avec `and $t4,$t4,$zero`.
 - `$t1` = compteur (taille du tableau).
 - `$t0` = adresse de base du tableau.
- **Boucle :**
 - Charger l'élément courant (`lw $t2,0($t0)`).
 - Ajouter à la somme partielle (`add $t4,$t4,$t2`).
 - Décrémenter le compteur (`addi $t1,$t1,-1`).
 - Incrémenter la base pour l'élément suivant (`addi $t0,$t0,4`).
 - Reboucler tant que `$t1 ≠ 0`.
- **Fin :**
 - Stocker le résultat dans `res` (`sw $t4,0($t0)`).
 - Retour au système (`jr $ra`).

Interruption et exceptions dans MIPS-R3000

Événements interrompant un programme :

1. Exceptions
2. Interruptions
3. Appels système (SYSCALL)
4. Réinitialisation (RESET)

1) Exceptions

- Surviennent pendant l'exécution d'une instruction lorsqu'une opération est impossible (ex. overflow, division par zéro).
- Types d'exceptions sur R3000 :
 - **ADEL** : adresse illégale en lecture
 - **ADES** : adresse illégale en écriture
 - **OVF** : dépassement de capacité pour ADD, ADDI, SUB
 - **RI** : codop illégal
 - **CPU** : coprocesseur inaccessible (instruction privilégiée)

Traitement d'une exception :

- Passer en **mode superviseur**
- Sauvegarder l'adresse de l'instruction fautive dans **EPC**
- Sauvegarder l'ancien **SR**
- Écrire le type d'exception dans **CR**
- Se brancher à **0x80000180** (gestionnaire d'exceptions)
- Pas de reprise de l'instruction fautive sur R3000

2) Interruptions

- Événements externes interrompant la tâche courante pour exécuter une routine.
- **Interruptions non masquées** : ne peuvent pas être interrompues
- **Interruptions masquées** : peuvent attendre
- R3000 : 6 sources d'interruptions matérielles
- Traitement d'une interruption :
 - Sauvegarder **PC+4** dans **EPC**
 - Sauvegarder **SR**
 - Passer en mode superviseur et masquer les interruptions
 - Mettre à jour **CR** pour indiquer la source
 - Se brancher à **0x80000180**

3) Appels système (SYSCALL)

- Permettent d'exécuter des fonctions système (ex. lecture/écriture).
- Pour exécuter un syscall :
 - Arguments dans **\$a0**, **\$a1**
 - Numéro de l'appel dans **\$v0**
 - Instruction **syscall**

Service	Code	Argument	Résultat	Commentaire
Print_int	1	\$a0 = int		Imprime sur la console l'entier transmis dans \$a0
Print_float	2	\$f12 = float		Imprime le réel transmis dans \$f12
Print_double	3	\$f12 = double		Imprime le réel transmis dans \$f12 sur 64bits
Print_String	4	\$a0 = string		Imprime la chaîne de caractère d'adresse transmise dans \$a0
Read_int	5		Entier dans \$v0	Lit un entier de la console et le met dans \$v0
Read_float	6		Réel dans f12	Lit un réel et le met dans \$f12
Read_double	7		Double dans f12	Lit un réel et le met dans \$f12 sur 64 bit.
Read_string	8	\$a0=buffer, \$a1=length		Lit un ensemble de caractères de longueur contenue dans \$a1 et met l'adresse dans \$a0
sbrk	9	\$a0=quantité	Adresse dans \$v0	Renvoie un pointeur vers un bloc de données de quantité donnée
Exit	10			Arrête le fonctionnement du programme.

Appels système pour entrées/sorties

1) Écrire un entier (**Print_int**)

- Mettre l'entier dans **\$a0**
- Numéro de syscall : 1
- Exemple :

```
li $a0,10 # charger l'entier 10
li $v0,1  # code syscall pour afficher un entier
syscall   # exécuter
```

2) Lire un entier (**Read_int**)

- Numéro de syscall : 5
- Résultat récupéré dans \$v0
- Exemple :

```
li $v0,5 # code syscall pour lire un entier
syscall # lecture dans $v0
```

3) Lire une chaîne (Read_string)

- \$a0 : adresse de début de la chaîne
- \$a1 : longueur maximum
- Numéro de syscall : 8
- Exemple :

```
li $v0,8
la $a0,buffer # adresse du buffer
li $a1,60 # taille max
syscall
```

4) Écrire une chaîne (Print_string)

- \$a0 : adresse de début de la chaîne
- Numéro de syscall : 4
- Exemple :

```
li $v0,4
la $a0,buffer # adresse du buffer à afficher
syscall
```

5) Quitter le programme

- Numéro de syscall : 10
- Exemple :

```
li $v0,10 syscall # termine le programme
```

Programme : Lire deux entiers et afficher leur somme

```
.data m1: .asciiz "Donnez le premier opérande : "
m2: .asciiz "Donnez le deuxième opérande : "
m3: .asciiz "Le résultat est : "
.text
.globl main
main: # Afficher le premier message
    li $v0, 4 # syscall print_string
    la $a0, m1 # adresse de la chaîne
    syscall
    # Lire le premier entier
    li $v0, 5 # syscall read_int
    syscall
```

```

add $t0, $v0, $zero # sauvegarder le premier entier dans $t0
# Afficher le deuxième message
li $v0, 4 # syscall print_string
la $a0, m2 # adresse de la chaîne
syscall
# Lire le deuxième entier
li $v0, 5 # syscall read_int
syscall
# Additionner les deux entiers
add $t0, $t0, $v0
# Afficher le résultat
li $v0, 4 # syscall print_string
la $a0, m3
syscall
li $v0, 1 # syscall print_int
move $a0, $t0
syscall
# Terminer le programme
li $v0, 10
syscall

```

4) Le signal RESET

- Le signal RESET force le processeur à exécuter un programme spécial de réinitialisation.
- Il agit comme une interruption non masquable, avec une adresse propre (0xBF000000).
- L'adresse de retour n'est pas sauvegardée.
- Lors de l'activation (au moins un cycle d'horloge), le processeur passe en mode superviseur et masque toutes les interruptions.

Chapitre 4 : Les procédures et les fonctions

1/Introduction

Les procédures servent à structurer les programmes.

- Une procédure est un sous-programme exécuté sur le même processeur que le programme appelant et partage les mêmes registres.
- Il faut préserver et restaurer le contenu des registres utilisés par la procédure.
- Les paramètres doivent être accessibles par la procédure et peuvent changer à chaque appel.
- À la fin de l'exécution, le contrôle doit revenir au programme appelant.

Étapes d'exécution d'une procédure :

1. Placer les paramètres à un endroit accessible par la procédure.
2. Transférer le contrôle à la procédure (appel).
3. Gérer correctement les ressources du processeur (registres, pile).
4. Effectuer le traitement demandé.
5. Placer les résultats à un endroit accessible par l'appelant.
6. Rendre le contrôle au programme appelant (retour).
7. Une même procédure peut être appelée depuis plusieurs points du programme.

2/Rôle de la pile en MIPS

- La pile est une zone mémoire de taille dynamique, utilisant le principe **LIFO** (Last In, First Out).
- Le registre **\$sp (R29)** pointe vers le sommet de la pile et permet d'y lire et d'y écrire.

Utilisations principales de la pile :

- Sauvegarde de l'adresse de retour lors des appels de procédures.
- Passage des paramètres entre procédures.
- Stockage des variables locales des procédures.
- Les processeurs utilisent la pile principalement lorsque les registres ne suffisent plus.
- L'architecture MIPS dispose de deux piles :
 - une pile pour les programmes utilisateurs,
 - une pile réservée au système.
- MIPS ne possède pas de pointeur de pile matériel dédié ; le registre général **R29 (\$sp)** est utilisé comme pointeur de pile.

3/Appel et retour de procédure :

- L'appel d'une procédure consiste à effectuer un saut vers sa première instruction.
- En MIPS-R3000, l'appel se fait avec l'instruction **jal procédure**.
- **jal** transfère le contrôle à la procédure **et sauvegarde l'adresse de retour** dans le registre **R31 (\$ra)**, et non dans la pile.
- Le retour de procédure se fait avec **jr \$ra**, qui provoque un saut vers l'adresse contenue dans **\$ra** (adresse de retour).

4/Passage des paramètres et valeurs de retour :

- Les **quatre premiers paramètres** d'une procédure sont passés dans les registres **\$a0 à \$a3**.
- Les paramètres supplémentaires sont transmis via **la pile**.
- Les **deux premières valeurs de retour** sont placées dans les registres **\$v0 et \$v1**.
- Les valeurs de retour supplémentaires sont également passées par **la pile**.

5/Intégrité des registres :

- Le programme appelant et la procédure utilisent **les mêmes registres**, ce qui peut provoquer des conflits si la procédure modifie des registres déjà utilisés par l'appelant.
- Exemple : si **\$S1** vaut 11 avant l'appel d'une procédure et que la procédure modifie **\$S1** pour mettre 19, après le retour, **\$S1** n'a plus sa valeur originale.
- Conséquence : les calculs effectués après le retour peuvent être incorrects si les registres n'ont pas été sauvegardés.
- **Règle** : une procédure doit **sauvegarder les registres \$S0 à \$S7** qu'elle utilise dans la pile avant modification et les **restaurer** avant de retourner au programme appelant.
- Les registres temporaires **\$t0 à \$t9** ne sont **pas sauvegardés**, car ils sont considérés comme volatils et peuvent être librement utilisés par la procédure.

Exemple 1 de programme MIPS

Code C à traduire :

```
void main() {  
    int x, y;  
  
    int blabla(int g, int h, int i, int j) {  
        int f;  
        f = (g+h) - (i+j);  
        return f;  
    }  
    x = 5;  
    y = x + blabla(1,2,3,4);  
}
```

Traduction en assembleur MIPS :

.data

x: .word 0 # variable globale x

y: .word 0 # variable globale y

.text

.globl main

Programme principal : main

main:

li \$S0, 5 # \$S0 = 5

 # x = 5

la \$t0, x # charger l'adresse de x

sw \$S0, 0(\$t0) # stocker 5 dans x

Préparation des arguments de la fonction blabla

li \$a0, 1 # g = 1

li \$a1, 2 # h = 2

li \$a2, 3 # i = 3

li \$a3, 4 # j = 4

jal blabla # appel de la fonction blabla

add \$S1, \$S0, \$V0 # $y = x + \text{blabla}(1,2,3,4)$

la \$t1, y # charger l'adresse de y

sw \$S1, 0(\$t1) # sauvegarder y en mémoire

jr \$ra # retour au système

Fonction blabla $f = (g + h) - (i + j)$

blabla:

addi \$sp, \$sp, -4 # réserver de l'espace dans la pile

sw \$S0, 0(\$sp) # sauvegarder le registre \$S0

add \$t0, \$a0, \$a1 # $t0 = g + h$

add \$t1, \$a2, \$a3 # $t1 = i + j$

sub \$S0, \$t0, \$t1 # $f = (g+h) - (i+j)$

add \$V0, \$S0, \$zero # placer la valeur de retour dans \$v0


```
lw $S0, 0($sp)    # restaurer $S0
```

```
addi $sp, $sp, 4   # libérer l'espace de la pile
```

```
jr $ra            # retour à main
```

♦ Commentaires importants (comme en cours)

- **\$S0 et \$S1**
→ utilisés dans le **programme principal** pour les variables **x** et **y**
- **\$S0**
→ utilisé dans la **fonction** pour la variable locale **f**
→ donc **obligatoirement sauvegardé dans la pile**
- **\$a0 à \$a3**
→ servent à passer les paramètres **g, h, i, j**
- **\$v0**
→ contient **la valeur de retour** de la fonction
- **jal blabla**
→ appelle la fonction
→ sauvegarde automatiquement l'adresse de retour dans **\$ra**
- **jr \$ra**
→ retourne au programme appelant

♦ Pourquoi on utilise la pile ici ?

Parce que la fonction **blabla** modifie **\$S0**
Or les registres **\$S** **doivent être préservés**
Donc :

1. Sauvegarde dans la pile au début
2. Restauration avant le retour

Exemple 2 : Appels imbriqués avec sauvegarde de pile

Code C :

```
int blabla_1(int x, int y) {  
    int i, j;  
    i = x * x;  
    j = i + y;  
    return j;  
}  
int blabla_2(int a) {  
    int n, m;  
    n = a + 5;  
    m = a + blabla_1(n, a);  
    return m;  
}
```

```

void main() {
    int w, z;
    w = blabla_1(1,2);
    z = blabla_2(10);
}

```

Traduction en assembleur MIPS :

```

.data
w: .word 0
z: .word 0
.text
.globl main
main:
    li $S0, 0
    li $S1, 0
    # Appel de blabla_1
    li $a0, 1
    li $a1, 2
    jal blabla_1
    add $S0, $v0, $zero    # w = valeur de retour
    # Appel de blabla_2
    li $a0, 10
    jal blabla_2
    add $S1, $v0, $zero    # z = valeur de retour
    # Sauvegarder w et z en mémoire
    la $t0, w
    sw $S0, 0($t0)
    la $t1, z
    sw $S1, 0($t1)
    jr $ra                # retour système
blabla_1:
    addi $sp, $sp, -8      # réserver la pile
    sw $S0, 0($sp)         # sauvegarder S0
    sw $S1, 4($sp)         # sauvegarder S1
    mult $a0, $a1          # i = x * x
    mflo $S0              # récupérer le résultat
    add $S1, $S0, $a1      # j = i + y
    add $v0, $S1, $zero    # retourner j
    lw $S0, 0($sp)         # restaurer S0
    lw $S1, 4($sp)         # restaurer S1
    addi $sp, $sp, 8       # libérer la pile
    jr $ra                # retour à l'appelant
blabla_2:
    addi $sp, $sp, -16     # réserver la pile
    sw $S0, 0($sp)         # sauvegarder S0
    sw $S1, 4($sp)         # sauvegarder S1
    addi $S0, $a0, 5       # n = a + 5
    sw $a0, 8($sp)         # sauvegarder a0
    sw $ra, 12($sp)        # sauvegarder l'adresse de retour

```

```

add $a1, $a0, $zero    # préparation de l'appel
add $a0, $S0, $zero    # $a0 = n
jal blabla_1           # appel de blabla_1
lw $a0, 8($sp)         # récupérer valeur originale de a0
add $S1, $a0, $v0      # m = a + blabla_1(...)
add $v0, $S1, $zero    # retourner m
lw $S0, 0($sp)         # restaurer S0
lw $S1, 4($sp)         # restaurer S1
addi $sp, $sp, 16      # libérer la pile
jr $ra                 # retour au programme appelant

```

Explications importantes :

- **Variables locales** : stockées dans `$S0` et `$S1`.
- **Paramètres** : `$a0-$a3`.
- **Valeur de retour** : `$v0`.
- **Pile (\$sp)** : utilisée pour sauvegarder les registres avant modification et restaurer après l'exécution de la fonction.
- **Appels imbriqués** : `blabla_2` appelle `blabla_1` et utilise la pile pour préserver son contexte.
- **Retour de fonction** : `jr $ra` saute à l'adresse sauvegardée dans `$ra`.